Backend Developer Coding Test

Use Case: Build a simple e-commerce platform for managing products.

Task 1: Laravel

Create a New Laravel Application:

- Set up a new Laravel application with user authentication (registration, login) and authorization levels (admin, user).

Implement Product CRUD Functionalities:

- **Create:** Allow adding new products with details like name, description, price, category, image.
- **Read:** Display a list of all products, searchable and filterable by category.
- **Update:** Edit existing product details.
- **Delete:** Remove unwanted products.

Database Interaction:

- Use Eloquent ORM for database interactions.
- Implement data validation for product inputs.

Unit Tests:

- Write unit tests for core functionalities like product creation, reading, updating, and deleting.

Task 2: RESTful API

Endpoints:

- `/products`: Get all products, filter by category or price range (GET).
- `/products/:id`: Get a specific product by ID (GET).
- `/products`: Create a new product (POST).
- `/products/:id`: Update an existing product (PUT).
- `/products/:id`: Delete a product (DELETE).

Authentication and Authorization:

- Implement proper authentication for API access using Laravel Passport or Sanctum.
- Ensure only authorized users (admins) can create, update, or delete products.

Design Principles:

- Follow RESTful design principles for consistent and predictable API behavior.
- Provide clear and concise API documentation.

Additional Considerations:

Data Management:

- Use mock data or create a local database for testing purposes if a real database is not provided.
- Ensure proper data validation and sanitization to prevent potential security vulnerabilities.

Code Quality and Best Practices:

- Write clean, maintainable, and well-documented code.
- Adhere to best practices and coding standards for PHP and Laravel.
- Follow Laravel's conventions and style guides for consistency and readability.

Testing and Error Handling:

- Implement thorough unit tests using PHPUnit or Pest to ensure code quality and prevent regressions.
- Handle errors gracefully, providing meaningful feedback to the user.
- Consider using logging mechanisms for debugging and monitoring purposes.

Optional Front-End Task (Vue.js)

If you have experience with Vue.js and want to demonstrate your front-end skills, you can complete the following additional task:

Vue.js Component:

- Build a Vue.js component that displays a list of products retrieved from the Laravel backend API.
- Implement product search and filtering based on category, price range, etc.
- Provide user interaction for adding, editing, and deleting products (restricted to admin users).
- Design a user-friendly and responsive interface with proper error handling.

Instructions:

This coding test is designed to assess your problem-solving skills, coding proficiency, and understanding of best practices in backend development. Please read the instructions carefully before starting your test.

Please send us a link to your GitHub repository containing the completed test. You may use any public or private repository of your choice. If you choose to use a private repository, please add the following GitHub user as collaborators: @zenner88